

Do Agent Benchmarks Do What They Say?

An Executable-Contract Audit of Tool-Using Agent Environments

Rohith Reddy
Florida International University
Miami, FL, USA

Zichong Wang
Florida International University
Miami, FL, USA

Wenbin Zhang
Florida International University
Miami, FL, USA

Abstract—Tool-using agents are entering settings where a wrong action carries real cost, and the benchmarks certifying them execute each call in simulation and grade what the call reports, assuming the tool did what its interface advertises. The audits we survey inspect tasks, gold solutions, and graders, never that assumption. A benchmark score is a published data product, and a defect beneath it is present on every rerun, so the exposure it creates travels with the number into every reuse. We treat each tool’s advertised surfaces as an executable contract, check the implementation against it, and trace which task verdicts read state a defective tool should have written. Across 34 mutating tools in four benchmarks we confirm eight defect instances at pinned commits, one headline-eligible class in each, meaning one visible to the agent. Against injected defects the checker raised no false positive in 25 flags pooled from toy and real sites; its recall is low, and 29 of 33 misses trace to the probe corpus that drives the tools, not the taxonomy. The clearest case is a clinical benchmark whose interface tells the agent each write executed, under a no-write design its paper documents and its interface does not. Its grader takes that message as evidence, so its action success rate records whether a request was well formed, not whether any record changed. Two repairs follow: disclose a simplification on the surface the agent reads, and ground a write grader in state rather than in the tool’s own success string.

Index Terms—large language models, autonomous agents, data quality, benchmarking, data provenance, conformance testing

I. INTRODUCTION

Agentic benchmarks help decide which language models ship, and the agents they power are moving into settings where a mistaken action carries a real cost: managing a patient record, executing a financial transaction, or committing a change to a production codebase. A benchmark scores an agent by executing its tool calls against a simulated environment and grading what those calls report having done, whether by reading the state they left behind or by reading the results they returned. Either way, the grade assumes that each call did what its interface said, and that assumption sits underneath every leaderboard position and model-selection decision built on such a benchmark’s number.

That assumption is fragile: LiveClawBench names, as its motivating problem, that agent-benchmark mocks are commonly “reduced to endpoint-level stubs that remove sessions, artifacts, state transitions, and downstream side effects” [1]. The benchmark audits we survey examine task instructions, gold solutions, evaluation scripts, environment configurations, graders, and judges [2], [3], [4], [5]. [Zichong: The revision still states that each audit stops one layer above the tool body and that none tests the interface-to-state-transition contract. The

absence of a dedicated taxonomy category does not establish that a method never examines the corresponding behavior. Limiting the statement to four cited audits reduces its scope, but does not resolve this difference between the taxonomy claim and the claim about what those methods inspect.] [Response: You are right that scoping the claim narrowed it without fixing it. The two claims needed separating, not shrinking. What we actually examined is their published category schemas, so that is now all the paragraph asserts: none carries a category for a success signal decoupled from the state it claims to have written, 27 categories mapped in the gate file. The sentence claiming those methods do not test the contract is gone, and the paragraph now says in so many words that a schema does not settle what a method would surface in practice. The layering sentence stays because it is about our object, not about their behaviour.] [Response: All three are now scoped to the four audits we cite, and the figure caption names the two categories it means. What remains beyond that scoping is narrower and checkable: none of those taxonomies has a category for a success signal decoupled from state, 27 categories mapped in the gate file, which is a claim about their published schemas rather than about what they would have found.] What we can check of these is their published schemas, and none carries a category for a success signal decoupled from the state it claims to have written (§ II, 27 categories); whether they would surface one in practice is not something a schema settles. The layering is what makes the question live: a grader inspecting sandbox state inherits any defect in the tool that wrote it. Figure 1 puts that layering on a single write: the audits we survey read what the agent is told and what the grader scores, and the contract between them goes untested.

[Zichong: The stated scope of contract validation is inconsistent with the response about MedAgentBench. The response explicitly says that no contract exists for this benchmark and that its classification comes from manual inspection. The introduction nevertheless states, “For each mutating tool we author an executable contract,” followed by static and dynamic checking. Because MedAgentBench is the central example, this wording suggests that it underwent the described contract-based validation. The statement that seven findings were discovered manually does not resolve this ambiguity: manual discovery and the absence of subsequent executable validation are different facts.] [Response: They are different facts and the introduction ran them together. It said we author a contract for each mutating tool and then check it statically and dynamically, which reads as

a claim about all four benchmarks including the one that has no adapter. That sentence now names the split: the checker runs on the three benchmarks with adapters, and MedAgentBench's clauses are read by hand, which the experiments section already repeated at each of its findings. Saying seven findings were manual was never an answer to this, and we should not have treated it as one.] [Response: We went back to their paper before changing anything. Section 2.4 says they send only GET requests so the environment does not need re-initialising, and 2.4.3 says a POST gets a JSON-loadable sanity check and is then reported to the agent as successful. That is a documented choice with a stated reason, so under our own criterion the missing write is not a defect. The paragraph says that now. What remains is what the criterion does catch. The tool returns an unqualified success string, nothing the agent can see discloses the design, and the write-task grader admits evidence only when it matches that string. One other claim rested on the old framing and is gone, the one about five prior audits, which our own gate file never supported. And because agent visibility now carries the finding, the paper says plainly that the validator's visibility check never ran on it: no contract exists for MedAgentBench, so that classification is a hand reading at the pinned commit.]

One shipped benchmark exhibits the divergence directly. MedAgentBench [6] evaluates language-model agents against a simulated electronic health record, and its paper documents a deliberate simplification: only GET requests reach the environment, and a POST instead receives a JSON-loadable sanity check after which the harness “indicate[s] success of execution to the agent system” (§2.4, §2.4.3 of that paper). Nothing the agent reads carries that disclosure. The prompt template states call syntax only, and the tool schemas are translated from FHIR, the clinical-records interchange standard. The branch handling every write parses the payload into a local variable and never reads it again (`src/server/tasks/medagentbench/__init__.py`, lines 85–91 at commit 9926011). It injects into the agent's transcript the unqualified string “POST request accepted and executed successfully.” No code in the repository performs the write (`git grep send_post_request` returns nothing at the pinned commit; `FINDINGS-VERIFIED.md`). The defect this paper reports is therefore not the missing write, which the maintainers chose and documented, but the agent-visible claim that it happened. The benchmark's own write-task grader then relies on that claim as its evidence of record (Fig. 1; § V).

We set out to test that interface-to-implementation-to-state-transition contract directly, for every mutating tool a benchmark ships, meaning every tool whose advertised surface claims a state change, delivered or not, and three challenges make that hard. **No contract exists to check against.** One must be authored from the tool's own advertised surfaces, and a check built on a surface that is itself wrong inherits the error. **Simplification is not defect.** A simulated environment is deliberately simplified in ways its maintainers consider harmless, so telling a disclosed simplification from an undisclosed one needs a principled rule, not a maintainer to ask. **A violation**

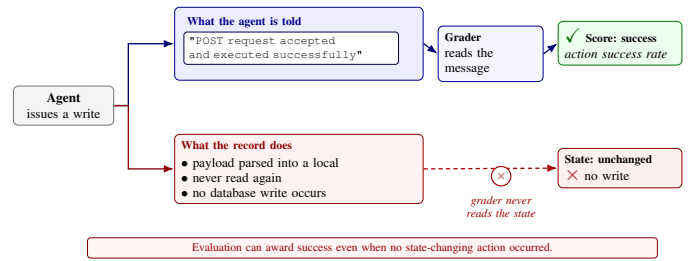


Fig. 1. One write in MedAgentBench, seen three ways at once. The agent is told the write executed, no record changes, and the grader scores the message rather than the state. The audits we survey read the blue path and the score; the red path, what the implementation actually did, is what this paper tests (§ II).

does not name the scores it reached. That requires tracing the defect through the benchmark's own evaluation code to the fields it reaches.

[Zichong: The added definition of a mutating tool also conflicts with the motivating failure case. The introduction defines these as tools that change environment state, while its central example concerns a tool that advertises a state change but performs none. Under the literal definition, such a tool falls outside the stated audit scope. This wording conflates the effect promised by the interface with the effect actually produced by the implementation, even though distinguishing those two is the central purpose of the paper.] [Response: This one is the root of the other two. We had written the scope in terms of what a tool does, when the paper exists to separate what a tool says from what it does, so the flagship example fell outside our own definition. Scope is now set on the advertised side: a tool is in scope when its advertised surface claims a state change, delivered or not. That is also the right side for the audit, since the contract is authored from that same surface, and the experimental setup now states the criterion it counts 34 under, including where advertisement and metadata would disagree.]

That untested contract is also a data-quality problem: benchmark scores are a published data product the field mines and reuses in leaderboards and model-selection decisions, the context the data-quality dimensions literature judges a product against [7]. This paper audits that provenance, formalized for a query by dependency-analysis research [8] and surveyed as a data-management problem in its own right [9]. A defect in that instrument is, unlike noise, present on every rerun, so the exposure it creates travels with the number into every downstream use. What that exposure could amount to is what score-at-risk bounds, without asserting that any verdict changed.

This paper meets those three challenges directly. For each mutating tool we author an executable contract from the tool's own advertised surfaces (docstring, schema, prompt text, return strings), and check implementation and state transitions against it with a static and dynamic checker on the three benchmarks that run under adapters. MedAgentBench has none and is read by hand, as § V notes at each of its findings. We then trace which task verdicts depend on the state the defect corrupted, tagging each bound by the evaluator basis that produced it. The contributions are these.

- 1) **Confirmed findings:** four headline-eligible benchmark-class defect cells across four shipped benchmarks, from eight verified instances confirmed at a pinned commit (§ V). The checker’s role here is confirmation and score-tracing rather than discovery: § V’s Surfaced-by column shows seven of eight findings surfaced manually and one by static check.
- 2) **Six executable defect classes**, defined as checker rules rather than prose (§ III). Phantom Effect and Partial Effect have no counterpart in any surveyed audit taxonomy.
- 3) **An executable contract-and-checker method**, in three parts (§ IV). A provenance-bound contract format, in which every clause cites the advertised surface it operationalizes, checked statically and dynamically against implementation and state transitions. A score-at-risk trace bounding how many task verdicts depend on state a defective tool should have written, each row tagged with its evaluator basis. And open-world validation of detection, precision and recall against taxonomy-shaped and off-taxonomy injected defects alike, under a pre-registered plan (§ V).

What the paper establishes is that four shipped benchmarks tell an agent things their implementations do not do, and that the verdicts those divergences could reach are traced per benchmark, each bound tagged or reported as undefined. The rest of the paper proceeds section by section: related work, notation and taxonomy, method, experiments, and conclusion.

II. RELATED WORK

Three questions organize the work around this paper: what a benchmark treats as ground truth when it scores a tool call, who has audited benchmarks and at which layer, and what the contract-testing literature already provides. [Zichong: The distinction between the first two subsections is unclear. The first discusses benchmarks that evaluate resulting state, while the second discusses verifying completion through that state. These topics are closely related, but the text does not establish a clear progression between them. Consequently, the second subsection initially feels like a continuation of the first rather than a separate research area.] [Response: You are right that the two read as one topic split in half. The cause was that the section was ordered by citation group, not by question, so nothing made the second follow from the first. Each subsection now opens with the question it answers, and the three questions chain: what a benchmark treats as ground truth, who has audited benchmarks and at which layer, and what the contract literature already provides.]

A. What a Benchmark Treats as Ground Truth

A benchmark that scores a tool-using episode has to decide what evidence settles the question. The state-based benchmarks we survey settle it with the state a call leaves behind. WebArena [10], AppWorld [11], OSWorld [12], and Android-World [13] grade by post-episode state, database rows, files, or device configuration, not the agent’s claim of success. [Zichong: The fairness measurement sentence interrupts the first

subsection’s flow. The paragraph moves from state-based task evaluation to tool-invocation disparity without explaining how this additional measurement setting relates to the preceding examples. Fairness is also not developed later, so the sentence reads as an isolated citation rather than part of the section’s argument.] [Response: Cut. It was a citation with no question to belong to, and nothing later developed it, which is exactly how it read.] tau2-bench [14] compares gold and predicted final database state or per-task assertion functions; AgentDojo [15] inspects the environment’s own post-call state. In each, the tool is the instrument and its output is the measurement, which makes the tool layer the thing a score ultimately rests on.

B. Audits of Benchmarks, and the Layer Each Reaches

If a score rests on the tool layer, the next question is who has checked it. [Zichong: Benchmark auditing does not have a clearly developed place in the section’s structure. BenchGuard, Automated Benchmark Audit, Tool-Veritas, and SafeAudit are among the closest works to this paper, but their discussion is distributed across a paragraph and a table. This gives the central auditing literature less organized treatment than the broader benchmark background.] [Response: Agreed, and this was the worst of it, because that is the literature closest to us. Benchmark auditing now has its own subsection instead of being split between a paragraph and a table, and it is organised by the layer each audit opens: task artifacts, verdicts, agent claims, and benchmark practice.] Benchmark auditing is the closest literature to this paper, and it divides by the layer each audit opens. Task-artifact audits read instructions, gold solutions, environment configuration and grader scripts: BenchGuard [2] across fourteen subcategories, Automated Benchmark Audit [3] across an instruction, environment and evaluation schema, and SafeAudit [5] across safety-suite coverage. Verdict-layer audits ask whether a grader’s decision matches the outcome: Tool-Veritas [4] on the same tau2-bench family we audit, Gao and Zhou [16] by showing a success criterion can accept a superficial proxy for the change it certifies, and JudgeSense [17] under prompt rewording. Agent-claim audits compare what an agent reports against what was recorded, in Advani [18] against tau2-bench and AppWorld state and in AgentProp-Bench [19] by measuring how often a corrupted parameter reaches a wrong final answer. Agent-Diff [20] moves the criterion itself onto a state delta read from a replica API, and a construct-validity review of 445 LLM benchmarks [21] asks the question one level of abstraction higher. [Zichong: The second subsection changes focus too frequently. It moves from Agent-Diff and completion verification to construct validity, then to the paper’s central example, evaluator sensitivity, and error propagation. These topics involve different research questions, but the transitions do not explain how each follows from the previous one. The subsection therefore lacks a stable organizing question.] [Response: That subsection had no organising question, so anything adjacent could be appended to it. Under the new question, which layer does each audit reach, the works you list either have a place in that order or leave. Construct validity stays because it is the same question one level up; our

own central chain leaves, for the reason in your next comment.] Benchmark-practice critiques sit alongside these: BetterBench [22] and UTBoost [23] audit reported practice systematically, Kapoor et al. [24] raise it as general practice without a taxonomy, and the Agentic Benchmark Checklist [25] reports τ -bench [26] counting empty responses as success.

Table I places the six closest on one basis: the layer each opens, and what it therefore takes on trust. [Zichong: The comparison table does not use a consistent basis across rows. Agent-Diff is compared through its evaluation mechanism, Tool-Veritas through the benchmark family it studies, BenchGuard and related methods through their taxonomies, and the Agentic Benchmark Checklist through its coverage period. Because each row addresses a different question, the table does not provide a clear comparison of how these works relate to the same research problem.] [Response: The table now asks one question of every row: which layer the work opens, and what it therefore takes on trust. That also makes the point the section is for, since every row trusts the tool implementation and that is the column a reader can scan.] Every row trusts the tool implementation, which is the layer none of them opens. [Zichong: The paragraph beginning A single-layer audit finds each layer consistent here shifts from reviewing prior work to defending the paper's own finding. References to the "central chain," the experiments, Ungrounded Oracle, and Phantom Effect require the reader to move between the literature discussion and the paper's technical claims. This interrupts the review and makes the paragraph read like part of a novelty argument or response to a reviewer.] [Response: Gone. That paragraph was arguing the paper's novelty inside the survey, and the argument already lives in the experiments section where the evidence is. What remains in the survey is the disposal, which is a statement about the other work rather than about ours.]

C. Contract Checking, and What Has to Change to Point It Here

The machinery for checking an implementation against a declared contract is mature; what it has not been pointed at is the tool-to-agent boundary. Design by contract with runtime verification is decades old [28], [29], and contract inference, oracle languages and invariant detection cover the surrounding ground: pointer-state API contracts [30], an executable contract language [31], REST invariants [32], frame specifications [33], and contracts for stateful modules [34] and deep-learning APIs [35]. Our Ignored Argument and Partial Effect rules are metamorphic relations in all but name [36], [37], [38]; stateful sequence exploration is established for REST APIs [39], [40]; and PolDet [41] and iDFlakies [42] anticipate Reset Leak through a comparable oracle, narrowing that class to a new target rather than a new technique.

A second line already applies this vocabulary to agents, and it consistently places the contract on the trusted side. Four such works [43], [44], [45], [46] check an agent against a contract taken as given, as does the 34-fault taxonomy [47], and the eight agent-code defect types of [48] are found in the agent application rather than in the environment it

is scored against. [Zichong: The third subsection begins with a recognizable technical theme, contracts, invariants, and API testing, but loses that focus in its second paragraph. Agent contracts, agent-code defects, benchmark guidelines, test isolation, and LiveClawBench are introduced in quick succession. The relationships among these groups remain unclear, making the paragraph read as a collection of remaining references rather than a connected account of the testing literature.] [Response: The second paragraph had become the place leftover citations went. It now has its own claim, that the agent-facing contract work consistently puts the contract on the trusted side, and every citation in it is an instance of that claim. Benchmark-practice critiques moved to the auditing subsection, which is where they were always about.] ToolFuzz [27] comes nearest in engineering terms, testing LangChain tools against their own documentation, but its oracle is the agent's response and the documentation is what it trusts. The technique therefore transfers and the target does not: we take the contract as the object under test rather than as the yardstick. [Zichong: The same novelty statement is repeated throughout the section: previous work trusts the tool, while this paper checks it. This repetition limits the progression of the discussion because successive groups of references repeatedly lead to the same conclusion without establishing distinct relationships to the proposed work. The final sentence about LiveClawBench also returns to the general motivation after the contract-testing discussion, leaving the section with an abrupt change of topic.] [Response: Each subsection now ends on a different relation rather than the same one three times: the first identifies the layer a score rests on, the second says no audit opens it, the third says the technique transfers but its target does not. The trailing LiveClawBench sentence is cut; the introduction already carries that motivation and it was returning the section to the start.]

III. NOTATIONS AND PRELIMINARIES

A benchmark tool is not application code that happens to be simulated: it is the write path of a measuring device. A defect in one is present on every rerun and reaches a published number silently if it reaches one at all, rather than filing an issue. The deterministic evaluators surveyed in § II read what the tool layer produced, whether as stored state or as the result a call returned, so a defective tool's output is inherited by any gate reading it. Consistency between a broken tool and its grader is not validity, and the tool layer needs its own audit.

a) *The tuple, and the defect taxonomy:* Every defect class this paper checks for is defined over the tuple (*pre*, *post*, *args*, *result*): the canonical state snapshots before and after a call, the call's arguments, and its return value. Table II defines six such classes, each an executable checker rule typed at the tool boundary; prose descriptions are glosses, the rule is the definition. All six were carved from the tau2-bench and MedAgentBench anchor findings before the other two were audited, so they fit those defects by construction, and the typing is a checking discipline, not a partition: Finding 5 trips Ignored Argument and Phantom Effect on one call.

TABLE I

THE SIX CLOSEST WORKS, ON ONE BASIS: THE LAYER EACH OPENS, AND WHAT IT THEREFORE TAKES ON TRUST. EVERY ROW TRUSTS THE TOOL IMPLEMENTATION, THE LAYER THIS PAPER TESTS.

Work(s)	Layer it opens	What it takes on trust
Agent-Diff [20] Gao and Zhou [16]	State delta across a replica API, as the success criterion The grading script, asking whether an outcome is backed by stored evidence	The replica API that reports the delta The tool that produced the evidence
Tool-Veritas [4] ToolFuzz [27]	Grader verdict against task outcome, on tau2-bench Retail Tool documentation against the agent’s response	The state the tool wrote The documentation, and the tool body
BenchGuard [2], Automated Benchmark Audit [3], SafeAudit [5] Agentic Benchmark Checklist [25]	Task artifacts, environment configuration, grader and safety coverage Reported benchmark practice, over a Jan. 2024–Mar. 2025 window predating tau2-bench	The tool implementation, unexamined The tool implementation, unexamined

No surveyed schema carries a category for a success signal decoupled from the state it claims to have written; GATE.md § 2 maps all 27.

TABLE II
SIX EXECUTABLE DEFECT CLASSES.

Class	Checker rule (over <code>pre</code> , <code>post</code> , <code>args</code> , <code>result</code>)	Status
Phantom Effect	$\text{success signal true} \wedge \text{advertised effect delta absent}$	field-observed
Unenforced Precondition	$\text{precondition predicate false} \wedge (\text{no error signal} \vee \text{state mutated})$	field-observed
Ignored Argument	$\text{post-state invariant under variation of an argument advertised as effective}$	field-observed
Partial Effect	$\geq 1 \text{ advertised effect predicate holds} \wedge \geq 1 \text{ fails on the same call}$	field-observed
Invariant Break	$\text{environment invariant false after a legal call sequence}$	mutation-only
Reset Leak	$\text{snapshot after reset} \neq \text{initial snapshot}$	mutation-only

Four of the six classes are field-observed in shipped benchmarks (§ V). Invariant Break and Reset Leak exist only as injected mutants, and every count in § V keeps that distinction visible. Ignored Argument was mutation-only until the third and fourth benchmarks, and is now the most common class, with four instances. § V’s open-world escape rate measures what that construction leaves uncovered rather than closing it.

b) Result-state disagreement is a signal, not a class: The Ignored Argument rule is deliberately state-only: a conjunctive rule would miss AgentDojo’s `reserve_car_rental`, which discards `end_time` from state while interpolating it into the success string (§ V, Finding 6). A result that varies where state does not is an aggravating signal, never a class.

c) The benign-simplification principle: The strongest objection a maintainer can raise is that these are intentional simplifications of a simulation that never claimed fidelity. Telling that case from an undisclosed divergence, the second challenge of § I, needs a criterion rather than a case-by-case judgment call, stated once and applied uniformly:

A simplification is benign exactly when it is advertised. The defect is never the simplification; it is the undisclosed divergence between what the interface tells the agent and what the implementation does.

Three consequences follow. First, the tier decides, not the intent. tau2-bench records the deferred flight-database update in a code comment (`airline/tools.py:689`) and the unreleased seats in a log line (367). Both are deliberate, both are maintainer-annotated, and neither reaches the agent, so neither clears the tool; the docstring at 689 never mentions the deferral. Second, either side of a divergence may be repaired, fixing a docstring as completely as fixing code; § V counts such a response as a resolved finding. Third, the advertised surface is the right baseline: the agent’s behavior is the measured quantity, scoped to **agent-side measurement validity**, audited throughout, not **consumer-side label validity**. That alternative is declined because a metric name has no fixed

audience to check a claim against, where agent-visible text is a discrete artifact. What clears a tool is disclosure the agent reads. MedAgentBench is the sharp case: a design disclosed in its paper and on no agent-visible surface is flagged by the same rule that would clear a docstring saying it (§ I, § V).

d) One evaluator-layer property, kept outside the taxonomy: MedAgentBench’s write graders exhibit a property we name Ungrounded Oracle. An oracle is whatever decides that a check passed; this one decides a state change by reading the actor’s claim rather than the state. Field-observed in § II, it is not a seventh class: the six classes are typed over (`pre`, `post`, `args`, `result`) at the tool boundary; oracle grounding is typed over grader source code instead. § V emits it as a per-task classification.

IV. METHOD

Fig. 2 lays out the pipeline this section describes, in order, and each stage answers one challenge from § I. Stage 1 is a contract specification authored against no pre-existing formal contract; stage 2 is a static and dynamic conformance checker applying § III’s benign-simplification criterion; and stage 3 is a score-at-risk dependency trace, tracing a confirmed defect to the verdicts it can reach. § V reports what each stage found.

A. The Contract Specification

Every contract clause cites the provenance of the advertised surface it operationalizes: file, line, and a verbatim quote checked at build time. Clauses fall into three grounding tiers: **agent-visible** text (docstring, schema, prompt template, tool return, README, external standard); **maintainer-annotated** text (a comment, log line, or TODO) no agent ever sees; and **inferred** semantics we assigned ourselves. A cell is headline-eligible only when its clause is agent-visible (§ III); the other two tiers are reported separately, never folded into a headline count. The benign-simplification principle of § III governs contract authoring too: a documentation fix resolves a finding

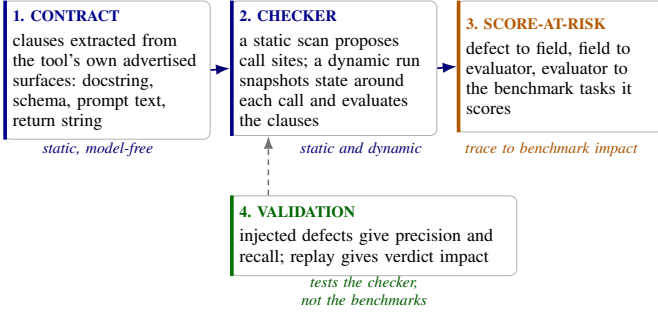


Fig. 2. Method overview. Stages 1 to 3 are static and model-free and match § IV’s three subsections in order. Validation is a branch rather than a step: it measures the checker, and § V reports what it found.

as completely as a code fix (§ V). The checker also knows when not to fire: tau2-bench’s deferred flight-database update and its non-enforcement of cancellation eligibility are both true negatives our test suite exercises. Contract authoring is itself measured by two annotator sets under a pre-committed probe corpus, a sensitivity analysis reported in § V.

B. The Conformance Checker

The checker’s static and dynamic halves are laid out in Fig. 2; UNTESTABLE stays in every denominator this paper reports, and the share is large: 96 of 180 clauses on tau2-bench, 11 of 60 on AgentDojo, 4 of 27 on MM-ToolSandbox. tau2-bench, AgentDojo, and MM-ToolSandbox run through adapters as subprocesses; MedAgentBench has no adapter and is checked statically (§ I). All six defect classes now have a real code path, but four did not until a post-freeze mutation sweep exposed the gap. Reset Leak and Invariant Break verdicts, 35 frame clauses, and 17 of 19 tau2 contracts were silently never dynamically exercised; the gap was fixed at the cost of one disclosed refreeze cycle (`checker-freeze-v1` to `checker-freeze-v2`, detailed once under Threats). Adapter mechanics are in the artifact.

C. Score-at-Risk as a Provenance Trace

This stage’s deliverable is not a violation count but a provenance trace run backwards: given a confirmed defect, which task verdicts could have been computed on state that defect corrupted. It asks of a score what dependency analysis asks of a query result: which inputs it depended on [8]. `analysis/score_at_risk.py` performs the trace over the 1,208 rows in `report/score_at_risk.jsonl`, no model in the pipeline; a `--verify` flag re-derives every row from the same pinned inputs and diffs it against the file on disk, checking reproducibility rather than claiming it. Three static steps:

- 1) **Defect to field.** From a contract’s VIOLATES clause, the predicate’s compiled abstract syntax tree, never its source string, yields the state paths the tool should have written and did not, or wrote unconditionally when a precondition should have blocked the write.
- 2) **Field to evaluator.** The benchmark’s evaluation code is parsed the same way, by AST rather than pattern match,

TABLE III
PER-BENCHMARK AUDIT TOTALS. CLASSES COUNTS EVERY LEDGER CELL, CONFIRMED INSTANCES AND THE ONE UNADJUDICATED CANDIDATE ALIKE; THE PARENTHEICAL IS THE HEADLINE-ELIGIBLE SUBSET.

Benchmark	Tools	Mut.	Impl.	Classes (HL)
MedAgentBench	3	3	1	2 (1)
tau2-bench	19	19	3	3 (1)
AgentDojo	7	7	3	2 (1)
MM-ToolSandbox	5	5	1	1 (1)
Total	34	34	8	8 (4)

Excluded (reported, not headline): MedAgentBench Ungrounded Oracle (evaluator layer); tau2-bench Ignored Argument (unadjudicated); tau2-bench Partial Effect (maintainer-annotated); AgentDojo Phantom Effect (fixture never trips biconditional).

for the state paths it reads, and each read site is classified by provenance, live simulator state or agent transcript.

- 3) **Evaluator to tasks.** Every task whose verdict depends on a field in that intersection is enumerated from the benchmark’s pinned task-definition file, exhaustively.

Every at-risk figure carries a basis tag and is never pooled across bases; § V’s detector-validation arm pools toy and real sites by design and is exempt. The bound runs in one direction only: at-risk is an over-approximation of misgrading via the state-dependency mechanism, holding only where the oracle is state-grounded. It says a verdict could have been computed wrong because its evaluator reads a field the defective tool controls, not that the verdict was wrong. § V reports the per-benchmark results of applying this trace, including whether the bound is vacuous on a drawn sample.

V. EXPERIMENTS

A. Experimental Setup

That method runs against four shipped agent benchmarks, each at a pinned commit: MedAgentBench [6] (clinical EHR); tau2-bench [14] (customer service, airline, retail and telecom domains); AgentDojo [15] (security-focused tool use); and MM-ToolSandbox [49] (multi-modal tool use). A tool is in scope when its advertised surface claims a state change, or the benchmark’s metadata marks it as mutating: tau2-bench’s `mutates_state` flag, which its own `toolkit.py` documents as authoritative over the `ToolType.WRITE` tag, and a hand read of the advertised surface elsewhere. The two coincide except at MedAgentBench’s write path, in scope on advertisement alone because its implementation changes nothing to mark. Scope is each benchmark’s agent-facing tool surface in the domains audited, three of tau2-bench’s five registered domains among them; within that scope no tool is sampled out. Every benchmark contributes at least one headline-eligible defect class, and three of the four also carry findings that do not clear the headline bar (Table III); instance-level evidence is in the findings ledger.

B. Baselines and Reference Points

A defect class none of the audits we survey checks for has no shared-task competitor, so we report four comparison points instead. **(1) Incumbent practice: manual source audit.** Table IV’s Surfaced-by column shows seven of eight findings

were found by a person reading source; the checker discovered one statically and reproduced three of the four headline cells dynamically. We do not claim to beat this baseline; we claim a precision-favoring flag (0 false positives over 25 mutation-arm flags) and a mechanical trace from a confirmed defect to the verdicts it can reach. **(2) Prior audits and the state-based graders of § II, by construction.** The surveyed task/grader-layer audits map zero of six defect classes across 27 categories (GATE.md § 2), and the state-based graders read the defective layer’s output as ground truth, so neither can flag it, which is a property of published designs rather than a performance claim. **(3) External precision reference:** AGORA+ reports 80% precision on REST invariant detection at 10K learning requests [32], cited for scale only. **(4) Injected-defect validation:** reported below; no tool-contract checker for these benchmarks appears in the work we survey, so the closest comparison is a system we would have had to build.

C. Evaluation Metrics

This paper’s evaluation vocabulary comes from mutation testing: inject a synthetic fault and check whether it is caught, treating surviving (“live”) mutants as a lower bound on what a contract misses [50]. The pre-registered plan fixes, before any mutant existed, what each arm can license (experiments/detector_analysis_plan.md, tag checker-freeze-v1, commit 9dacc79). The closed-world arm licenses “the checker detects violations of the clauses we wrote,” never “tool-contract defects”; the open-world arm licenses only a lower bound on the taxonomy’s incompleteness, never coverage. Every recall rate below is a design-effect-adjusted Wilson lower bound (Kish design effect, Rao-Scott one-way ANOVA ICC estimator, § 5 of the plan) with its effective denominator beside it. Toy-domain recall never enters a headline; headline recall is real-tools only, and precision pools both by design.

D. Implementation Details and Data Availability

The checker is frozen at checker-freeze-v2 (commit 54b74d4); every number below is scored under that freeze, not the superseded checker-freeze-v1 (9dacc79, § IV). The gold-trajectory replay below uses seed 1, zero trajectories discarded. Contract authoring followed experiments/annotation_protocol.md (commit fc776b6), with the cross-annotator comparability rule landing in the same commit as the reported result (dee7170), disclosed below as applied post hoc. The artifact (both pre-registered analysis plans, the checker, and every generated table) is archived at Zenodo, DOI 10.5281/zenodo.22182792, with development history at <https://github.com/rohithreddybc/tool-contract-conformance> (MIT license); audited benchmarks are pinned by upstream commit in FINDINGS-VERIFIED.md, not redistributed. Two reproduction paths ship with it. The default, make reproduce-results (or python reproduce.py where make is unavailable), re-validates every contract, re-derives each generated table and diffs it against the committed

copy, and audits every numeric claim in this paper. It also runs the test suite, with no network access, API keys, or model calls. The provenance and agent-visibility checks read upstream source, so make validate-full additionally requires the four benchmarks cloned under repos/ at their pinned commits.

E. Main Results

a) *Confirmed findings:* Across four shipped benchmarks, the ledger’s twelve VIOLATES clause rows collapse to eight confirmed finding instances, populating seven benchmark-class cells; the unadjudicated suspend_line candidate (below) adds an eighth, matching Table III’s Classes total. The compressed count is deliberate: the reporting rule counts unique defect classes per benchmark, since eight bugs in one copied helper is one bug. Four of the eight cells cannot carry a headline, for the reasons Table III’s footnote gives; all four are reported in full, labelled, and excluded. Every finding was verified by direct source reading at a pinned commit, independent of whoever surfaced it. Table IV’s **Surfaced by** column reports origin honestly: seven manual, one static check, none dynamic check, since reproducing a manual find is confirmation, not discovery. The dynamic checker reproduces three of the four headline cells as VIOLATES: tau2-bench’s Unenforced Precondition, AgentDojo’s Ignored Argument, and MM-ToolSandbox’s Ignored Argument. MedAgentBench has no adapter, so its cell is established statically instead.

b) *MedAgentBench, the central chain, and the numbers above it:* Finding 1 is the unqualified execution claim of § I: the POST branch performs no write. MedAgentBench’s paper discloses this design to its readers, but its interface never discloses it to the agent, so the finding is the agent-visible claim, not the omission. Finding 4 makes the omission unobservable from inside the benchmark: the grader reconstructs each write from the transcript, admitting evidence only when it matches Finding 1’s success string, so no FHIR read ever verifies the write. No contract exists for this benchmark under spec/contracts/, so Finding 1’s grounding rests on direct source reading, not validator check 8, recorded in FINDINGS-VERIFIED.md at the pinned commit above. Finding 4’s evidence file, refsol.py, is absent from the repository; we pin its SHA-256 in the artifact rather than redistribute a gated file, letting a reader verify the 60/90/150/0 classification below independently. MedAgentBench’s paper reports “Action SR”, the write-task success rate, for all 12 evaluated models in its Table 3 [6]: it ranges 0.00% to 71.33%, with the best score from Gemini-1.5 Pro. The paper’s abstract headlines 69.67% overall SR for Claude 3.5 Sonnet v2, a weighted blend of Query SR and this same Action SR (arXiv table, read directly; published in NEJM AI, vol. 2, iss. 9). The file containing the POST branch is unchanged at our pinned commit, touched only once, so every Action SR value was produced under an implementation in which no write occurs. The paper’s own § 2.4.1 describes “rule-based sanity checks to verify the correctness of the payload of POST requests,” i.e. the transcript-reconstruction mechanism above.

TABLE IV
CONFIRMED FINDINGS (INSTANCE LEVEL), WITH QUOTED EVIDENCE. EACH ROW IS CHECKABLE AT ITS PINNED COMMIT FROM THE QUOTED STRING.

#	Bench.	@ com- mit	Tool : line	Class	Tier	Surfaced by	Quoted evidence and disposition
1	MedAgentBench	@ 9926011	POST __init__.py:85-91	branch, Phantom Effect	agent-visible (prompt_ template, tool_return)	manual	“...executed successfully” fabricated; payload never re-read.
2	tau2-bench	c3398666	@ refuel_data, telecom/tools.py: 607-657	Unenforced Precon- dition	agent-visible (docstring)	manual	“must be Active” check commented out (selective disable).
3	tau2-bench	c3398666	@ cancel_ reservation, airline/tools.py: 315, 363-368	Partial Effect	maintainer- annotated: not headline-eligible	manual	“Seats release not implemented...!!!” (367); TODO (689). Seats never restored; per-episode reset (Reset Leak withdrawn).
4	MedAgentBench	@ 9926011	write graders, refsol.py (SHA- 256-pinned; not in repo)	Ungrounded Oracle (evaluator property, § III)	grader source	manual	“POST request accepted” gates extract_posts, graded from transcript, not live state.
5	AgentDojo	@ 089ed46	update_scheduled_ transaction, banking_ client.py:115-151	Ignored Argument + Phantom Effect	agent-visible (docstring)	manual	recurring guarded by truthiness (True only); returns “...updated” unconditionally.
6	AgentDojo	@ 089ed46	reserve_car_ rental, travel_ booking_ client.py:382-400	Ignored Argument	agent-visible (docstring)	manual	end_time dropped from state, kept in success string. No task exercises this tool.
7	MM- ToolSandbox	@ 1e8e932	venmo_social, mini /venmo.py:464-470	Ignored Argument	agent-visible (docstring)	manual	sort_by documented twice, never forwarded (listing branch; read-only, so the state test is vacuous and the evidence is the source).
8	AgentDojo	@ 089ed46	invite_user_to_ slack, slack.py: 93-103	Ignored Argument	agent-visible (docstring)	static check	“should be sent” to user_email; body never reads it.

A third tau2-bench candidate stays below the reporting threshold: suspend_line’s arg.reason VIOLATES at the inferred tier. A required reason is logged and never persisted, but whether the docstring advertises persistence rather than a log entry is contested; not counted among the eight confirmed instances, though it is the eighth cell in Table III’s ledger total.

The numbers are therefore exactly what that section describes: whether the agent emitted a well-formed POST request, not whether any clinical record changed. The disclosure sits in the paper’s evaluation section, not in the label “action success rate” a leaderboard reader sees, which is a consumer-side point outside the criterion audited here. tau2-bench’s pinned commit sits inside the lineage producing its published leaderboard, and both audited defects have been present, unpatched, across that span [14] (EXTERNAL-VERIFICATION.md Task 2). We claim no causal link from either defect to any specific published number.

c) *Score-at-risk*: Each defect carries its own bound and the evaluator basis behind it, kept side by side because the bases do not pool (Table V). tau2’s airline domain grades cancel_reservation by hashing the full database between gold and predicted runs (toolkit.py:242-244); any field a defective tool writes is read by that mechanism, so all 50 airline tasks come back at risk. That measures the evaluator’s coarseness, not the defect’s reach, so Table V tags it whole_state_hash instead of a sharper number. Telecom grades refuel_data through per-task assertion functions instead, and the intersection is exact: 1,135 of 2,285 tasks read a field the defective precondition lets through unconditionally, 1,120 on the written attribute (exact_field) and 15 more on the collection alone (collection_only). The figure of 2,285 is the full generated task file, not the roughly 114-task subset tau2-bench’s paper evaluates. A single benchmark carries both bases on different tools; no project-wide percentage is stated, only per-basis counts.

A naive field-intersection returns the wrong answer for

MedAgentBench, because the grader-checked field and the field the tool should have written share a name. A names-only rule would therefore call nearly every one of the 300 cases at risk. Classification is instead computed per grader function. Of 300 write-tagged cases, 60 are graded unconditionally from the transcript, 90 gate on live state but still grade the write from the transcript, 150 have no write oracle, and zero are graded by reading the write back from FHIR state. Score-at-risk is undefined for the 60 and indeterminate for the 90, never zero: “oracle not state-grounded.”

The experiment frame need not nest inside the at-risk population, and for Finding 5 the two sets are disjoint. update_scheduled_transaction is broken on every argument at falsy values, but recurring is the one argument for which a falsy value is legitimate, so it alone carries task risk. Four tasks’ gold solutions vary amount or recipient instead, landing in the experiment frame but not the at-risk set. The one task whose oracle reads .recurring, UserTask6, reaches that field through a different, non-defective tool, so the two rules select non-nested populations. No shipped travel task calls reserve_car_rental at all, so that row too is a field-name match through the oracle, and the findings ledger excludes it from score-impact claims on that ground. Both AgentDojo rows bound what the trace rule selects, not what the defect reaches. The rule is pre-registered and is not re-specified here after seeing which rows it returns.

MM-ToolSandbox reports what it can compute and nothing it cannot. Step 1 identifies the sort_by field venmo_social drops (Finding 7). Steps 2 and 3 do not run: AppWorld-tier scenarios exercising

TABLE V
SCORE-AT-RISK BY DEFECT, WITH BASIS AND GROUNDING. BASES NEVER POOL, AND NO ROW ESTABLISHES THAT A VERDICT WAS WRONG.

Benchmark : tool	Defect class	Basis	At risk / total	Experiment	Oracle	grounding frame
tau2 airline : cancel_reservation	Partial Effect	whole_state_hash	50 / 50	7		state_grounding
tau2 telecom : refuel_data	Unenforced Precondition	exact_field (1120) + collection_only (15)	1135 / 2285	1120		state_grounding
AgentDojo banking : update_scheduled_transaction	Ignored Argument	exact_field	1 / 16	4		state_grounding
AgentDojo travel : reserve_car_rental	Ignored Argument	exact_field	1 / 20	0		mixed
MedAgentBench : post-write	Phantom Effect	transcript (not state)	undefined	n/a	60 transcript / 90 mixed / 150 no-oracle / 0 state (of 300)	
MM-ToolSandbox : venmo_social	Ignored Argument	not computable	not_computable_appworld_unreachable	not computable		not computable

TABLE VI
REAL-TOOLS CLOSED-WORLD RECALL BY MUTATION OPERATOR.

Operator	n drawn	eff. n	de- tected	missed	untest- able	error	Recall (Wilson LB)
M-PHANTOM	7	7	5	0	0	2	≥ 0.359
M-PRECOND	7	5	1	6	0	0	≥ 0.020
M-IGNARG	21	15.13	4	12	1	4	≥ 0.066
M-PARTIAL	8	6	1	7	0	0	≥ 0.018
M-INVAR	8	8	0	8	0	0	≥ 0.000
M-RESET	n/a	n/a	n/a	n/a	n/a	n/a	no data

Eff. n is the tool-clustered effective sample size the design-effect adjustment computes from n drawn (§ 7 of the plan); it, not n drawn, feeds every Wilson bound above.

this tool are graded inside the appworld package, which this project cannot clone. Table V’s row reads not_computable_appworld_unreachable, not a zero: a null population is data about what a static, offline audit can reach, not a claim that no task is at risk.

d) *Real-tools closed-world recall:* Recall is low throughout (Table VI, each operator named for the class it injects), and a post hoc diagnostic, experiments/decompose_closed_world_misses.py, traces why. Of all 33 real-tool misses, the taxonomy itself owns 2 (clause_gap), the frozen probe corpus owns 29 (probe_corpus_unreachable, never driving the mutated path into an observable state), and the remaining 2 are probe_gap and target_not_found (detail is in the artifact). M-INVAR’s 0.000 reads as “no probe ever reached a mutated invariant path,” not “the checker cannot detect invariant breaks.” M-RESET has no real-tool site to draw at all: no tool-implementation file in the four benchmarks audited here defines a state-restoring function, so the honest report is “no data,” not a rate of zero. Biconditional success (the mechanism M-PHANTOM depends on) is opt-in per contract, adopted on 18 of 19 tau2-bench, 1 of 7 AgentDojo, and 0 of 5 MM-ToolSandbox contracts; where not adopted, Phantom Effect cannot fire at all, so the operator is drawable on fewer sites than the taxonomy suggests.

e) *Precision:* The checker raised zero false positives against any injected mutant in the closed-world arm, across a pooled toy-plus-real sample of 25 flags (TP=25, FP=0; 24 semantics-preserving controls scored alongside). Precision is therefore ≥ 0.867 at the Wilson lower bound. This denominator

is mutation-arm flags only; it says nothing about the field findings above, which were confirmed by direct source reading, not by this count. This number licenses nothing about recall, and unlike the AGORA+ reference point it structurally cannot false-positive against a clause it never wrote. That pooled sample also is not the pre-registered one: only M-PRECOND and M-IGNARG met the ~ 25 -per-class target, counting real and toy sites together, unlike the recall table’s real-tool-only n-drawn column of 7 and 21 above. The shortfall for the rest is detailed under Threats.

f) *Open-world escape rate, toy:* the cosmic-ray mutation tool scored 368 mutations across 4,048 (mutation, tool) pairs; 229 were behaviourally live, 225 of them escaped the checker, and the missed share is a Wilson lower bound of 0.956 (point estimate 0.983). Those 225 decompose as 119 tracing to a clause the taxonomy never wrote, 86 to a clause whose probes never drove the tool into the exposing state, and 20 to the checker crashing before evaluation. Only the mechanical declared-volatile-field rule ran in this automated pass, making 0.956 conservative toward overstating the checker’s blindness: a lower bound on the taxonomy’s incompleteness, not evidence against the closed-world numbers above.

g) *tau2-telecom produced no usable escape rate:* cosmic-ray scored 465 mutations across 2,325 (mutation, tool) pairs against the domain’s five non-anchor tools, and zero were behaviourally live, so the escape-rate denominator is zero. The cause is the probe corpus, not the detector: its recorded-real-call component does not exist, and the two placeholder components fail tau2’s entity-keyed checks before any mutation could show. A zero denominator is not evidence of a well-covered taxonomy; this arm is an open question. A precompute cache predating checker-freeze-v2 had produced a more favorable v1 run (281 of 465 scored, 33 live, escape rate ≥ 0.325), not reported since whether those 33 live mutants were genuine or cache artifacts is unresolvable.

h) *Evaluation impact: a null, not a gap:* Two tau2-bench findings carry mechanically verified one-hunk patches: cancelling a reservation leaves seats_after at 0 unpatched and restores it to 3 patched; refuelling a suspended line succeeds unpatched and raises ValueError

patched. Ten trajectories were replayed from each finding’s own gold reference solution, five per finding (seed 1, zero discarded). F3’s frame holds seven tasks, so five of seven are covered. Agent trajectories were unavailable offline (no model credentials), so gold references were substituted instead, a disclosed deviation from the pre-registered plan (`experiments/analysis_plan.md` §§ 4–5). All ten land in a single outcome cell (at risk, not exercised), and zero trajectories trip either trigger predicate, so zero verdicts flip. For F3 (airline `cancel_reservation`), no flip was possible by construction: gold-run and agent-run state execute the identical, still-defective tool. For F2 (telecom `refuel_data`), a flip was possible in principle and did not occur, because no gold trajectory calls the tool on an inactive line, and all 1,120 telecom tasks whose gold actions call it issue the same one. Per the pre-registered interpretation, this is a finding about conditional validity, not a null result: the score is correct today because no gold trajectory takes the broken path, and nothing in the benchmark prevents an agent from doing so. What that does and does not license is taken up under Threats. Full experiment detail is in the artifact.

i) Sensitivity analysis: contract-authoring agreement:

Two annotator sets wrote contracts for the same six tools against one pre-committed probe corpus. Both are agent sessions on one model family and only B wrote blind, so what this measures is whether a sentence turns into the same predicate twice, not whether two people would agree. This is evidence of stability in sentence-to-predicate translation under one-way blindness, not independent human judgment. The two sets are compared against a shared, pre-committed probe corpus across six tools, over comparable clause pairs on successful calls only. This scoping rule lands in the same commit as the result (`dee7170`) rather than in the protocol itself (`fc776b6`), disclosed here as applied post hoc. It drops `refuel_data` to $n=2$ of 9 probes and `reserve_car_rental` to $n=2$ of 11 on most pairs. The blind stratum agrees on 98 of 98 probe comparisons across 16 clause pairs and four of five blind tools; the fifth, `venmo_social`, contributes zero comparable pairs, a segmentation artifact that also cost it Finding 7’s own anchor `arg.sort_by`. Four tools needed hand-built witness probes, disclosed before any outcome was inspected, and the sixth, `cancel_reservation`, forms a separate contaminated stratum (Annotator B had read a worked example beforehand), never pooled with the blind result. One finding flips within the blind stratum and is not among the 98, since the comparability filter excludes its clause pair. The protocol’s own rule counts UNTESTABLE against anything as a disagreement rather than a skip, so the unfiltered count is 98 of 99. `reserve_car_rental`’s `arg.end_time` is VIOLATES under A and UNTESTABLE under B, a missing ISO-string annotation in B’s contract rather than a semantic disagreement (mechanism in the artifact).

F. Threats to Validity

Ten limits bear on the conclusions, ranked at the end of this section.

Taxonomy coverage. Six classes are not the space of possible defects. The toy-domain mutation arm measures how much escapes them, and § V reports that bound together with the clause-level decomposition of what got past.

Real-benchmark escape rate. The equivalent real-benchmark run produced no usable escape rate at all, for the reason § V gives with the counts: the probe corpus rather than the detector.

Sample size shortfall. Four of six operators fall short of the pre-registered target because the corpus is structurally small, not because of a scoring choice (§ V). Achieved N is reported beside the target throughout, never absorbed into a rate.

No real-tool reset site. Reset logic lives in the adapter layer, which rebuilds environments wholesale, so none of the four benchmarks audited here colocates reset with its tools. M-RESET therefore has no data rather than a recall of zero.

Subprocess exception boundary. The subprocess boundary absorbs exceptions the checker never sees, and whether that matters is unverified. A toy-domain bug in the closed-world v2 run showed it directly: an adapter’s `invoke()` caught only its declared error type, so a probe past a deleted guard drove the tool into code assuming the check held and the raw exception aborted the mutant instead of registering a violation. The same boundary sits between the checker and every real adapter, swallowing exceptions before the harness can classify them; an absorbed crash’s detected-vs-silent status is unchecked, bounding every real-tools recall number above.

Replay covers gold behaviour only. All 1,120 tasks whose gold actions call F2’s tool issue one identical call, so F2’s replays exhaust one identical gold call rather than sampling 1,120, and F3 covers five of its seven frame tasks (§ V). What the null cannot show is what an agent would do.

Anchor-driven benchmark selection. Benchmark selection is anchor-driven, not a sample of a population. `tau2-bench` and `MedAgentBench` entered this study because their defects were known before the four-benchmark protocol existed; `Agent-Dojo` and `MM-ToolSandbox` were added afterward to test whether the taxonomy generalized beyond the two anchors. No prevalence claim, what fraction of shipped benchmarks carry a defect class or how common Ignored Argument is, is supportable from four benchmarks chosen this way.

Frame-grammar expressiveness gap. The frame-path grammar cannot express one of its own most natural clauses. `spec/GRAMMAR-GAPS.md` records that no wildcard segment can exclude a single key, so a clause meaning “no other reservation is modified” cannot be written without colliding with the tool’s own advertised effect on the reservation it changes. The shipped `cancel_reservation` contract substitutes a weaker, expressible claim about an unrelated collection; the gap is a real coverage hole in the frame mechanism.

Repository history limits. `git init` ran on 2026-08-21, after most of the design work existed, and the first commit imported the project as one tree. A squashed import cannot demonstrate that any pre-registration preceded the decisions it was meant to freeze, and the paper does

not claim it does. What remains verifiable from commits after the import: `checker-freeze-v1` predates the first scored mutant, and `experiments/analysis_plan.md` predates the first recorded trajectory. The distinction is stated here rather than left to be inferred (`PROVENANCE.md`).

Disclosed refreeze cycle. One refreeze is disclosed rather than folded into the numbers: §IV details four taxonomy claims a sweep against `checker-freeze-v1` found uncoded, all fixed before `checker-freeze-v2` scored any number above. Ranked by how much each could move the conclusions, the missing recorded-real-calls corpus matters most, the subprocess boundary’s absorbed-exception ambiguity comes second since it bounds every recall number, and the anchor-driven selection third since it forecloses any prevalence claim.

VI. CONCLUSION

This paper tested a layer the benchmark audits we survey do not reach: the tool implementation that produces what a grader reads, and so the provenance of every score built on it. Four benchmarks yielded verified defects across four of six executable classes, and the score-at-risk analysis traced what those defects could have cost in published verdicts, tagged by evaluator basis rather than pooled. None of this establishes how common such defects are: anchor-driven audits are existence proofs, not a survey, and an at-risk bound is a due-diligence flag, not a correction to any published score. Maintainers can act on it now. Ground every write grader in stored state rather than in the tool’s own success string, and run a precision-favoring check against the tool layer, not only against tasks and gold solutions. Disclose every deliberate simplification where an agent reads it, since on that surface an undisclosed one is indistinguishable from a bug. A benchmark’s published numbers are a data product, and they are only as trustworthy as the layer those audits leave untested.

a) *Use of AI tools:* Per IEEE policy on generated content: generative AI assisted drafting, wrote the checker and analysis code, and produced the annotator sets and numeric analyses in §IV and §V. It enters no part of the evidence: every finding was confirmed by hand against upstream source at its pinned commit. The authors verified all content and are responsible for it.

b) *Coordinated disclosure:* All findings go to their maintainer teams before submission, with per-finding reproduction commands and proposed repairs, except MM-ToolSandbox, which disables issues and declines external reports in its `SECURITY.md`. The reporting rule is fixed in advance: a response received by submission is reported with its substance, a team that has not responded is recorded as such with the date and no inference drawn from silence, and a contested finding is marked contested with the maintainer’s reasoning. Responses are recorded in `report/disclosure_log.md`.

REFERENCES

- [1] Long, Du, Xu *et al.*, “LiveClawBench: Benchmarking LLM agents on complex, real-world assistant tasks,” 2026, arXiv:2604.13072.
- [2] Tu, Wang *et al.*, “BenchGuard: Who guards the benchmarks? automated auditing of LLM agent benchmarks,” 2026, accepted, Conference on Language Modeling (COLM) 2026.
- [3] Wang, Bianchi *et al.*, “Automated benchmark auditing for AI agents and large language models,” 2026, arXiv:2605.26079.
- [4] Bhat, Vaghasiya, Mohsin, and Aali, “Benchmarking the benchmarks: A validity audit of tool-calling evaluation,” 2026, arXiv:2607.02577.
- [5] Chen, Yan, Zhang, and Zhang, “Who tests the testers? systematic enumeration and coverage audit of LLM agent tool call safety,” 2026, arXiv:2603.18245.
- [6] Jiang, Black *et al.*, “MedAgentBench: A virtual EHR environment to benchmark medical LLM agents,” *NEJM AI*, vol. 2, no. 9, 2025, arXiv:2501.14654.
- [7] Wang and Strong, “Beyond accuracy: What data quality means to data consumers,” *Journal of Management Information Systems*, vol. 12, no. 4, 1996.
- [8] Buneman, Khanna, and Tan, “Why and where: A characterization of data provenance,” in *International Conference on Database Theory (ICDT)*, ser. Lecture Notes in Computer Science, vol. 1973. Springer, 2001.
- [9] Simmhan, Plale, and Gannon, “A survey of data provenance in e-science,” *ACM SIGMOD Record*, vol. 34, no. 3, 2005.
- [10] Zhou, Xu, Zhu *et al.*, “WebArena: A realistic web environment for building autonomous agents,” 2024, ICLR 2024.
- [11] Trivedi, Khot, Hartmann *et al.*, “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” 2024, ACL 2024 (Camera Ready).
- [12] Xie, Zhang, Chen *et al.*, “OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments,” 2024, NeurIPS 2024 D&B Track.
- [13] Rawles, Clinckemaillie, Chang *et al.*, “AndroidWorld: A dynamic benchmarking environment for autonomous agents,” 2025, ICLR 2025.
- [14] Barres, Dong *et al.*, “ τ^2 -bench: Evaluating conversational agents in a dual-control environment,” 2025.
- [15] Debenedetti, Zhang, Balunović, Beurer-Kellner, Fischer, and Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” 2024, NeurIPS 2024, Scopus-confirmed; D&B track sub-label unconfirmed.
- [16] Gao and Zhou, “Can agent benchmarks support their scores? evidence-supported bounds for interactive-agent evaluation,” 2026, preprint; no venue claim on the abstract page.
- [17] Bellilbatlu, Raff, and Zhang, “JudgeSense: A benchmark for prompt sensitivity in LLM-as-a-judge systems,” 2026.
- [18] Advani, “From confident closing to silent failure: Characterizing false success in LLM agents,” 2026, accepted, FAGEN workshop at ICML 2026 (non-archival); independently confirmed.
- [19] B. Gurram, “Auditing automated evaluation, error propagation, and runtime mitigation in tool-using language agents,” *arXiv preprint arXiv:2604.16706*, 2026, v2, 2026-08-10; preprint, no venue claimed.
- [20] Pysklo, Zhuravel, and Watson, “Agent-Diff: Benchmarking LLM agents on enterprise API tasks via code execution with state-diff-based evaluation,” in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2026.
- [21] Bean, Kearns, Romanou *et al.*, “Measuring what matters: Construct validity in large language model benchmarks,” 2025, NeurIPS 2025 Datasets and Benchmarks Track; not independently confirmed.
- [22] Reuel, Hardy *et al.*, “BetterBench: Assessing AI benchmarks, uncovering issues, and establishing best practices,” in *NeurIPS Datasets and Benchmarks Track*, 2024.
- [23] Yu, Zhu, He, and Kang, “UTBoost: Rigorous evaluation of coding agents on SWE-bench,” in *Proceedings of ACL*, 2025.
- [24] Kapoor, Vaithilingam *et al.*, “AI agents that matter,” *Transactions on Machine Learning Research (TMLR)*, 2024.
- [25] Zhu, Jin, Pruksachatkun *et al.*, “Establishing best practices for building rigorous agentic benchmarks,” 2025, v5. Preprint. Referred to in text as the Agentic Benchmark Checklist.
- [26] Yao, Shinn, Razavi, and Narasimhan, “ τ -bench: A benchmark for tool-agent-user interaction in real-world domains,” 2024, preprint; distinct artifact and codebase from τ^2 -bench [14] (§II).
- [27] Milev, Balunović, Baader, and Vechev, “ToolFuzz – automated agent tool testing,” 2025.
- [28] Karaorman and Abercrombie, “jContractor: Introducing design-by-contract to Java using reflective bytecode instrumentation,” *Formal Methods in System Design*, vol. 27, no. 3, 2005.
- [29] Bartetzko, Fischer, Möller, and Wehrheim, “Jass – Java with assertions,” *Electronic Notes in Theoretical Computer Science (ENTCS)*, vol. 55, no. 2, 2001.

- [30] Deng, Ma, Gao, and Sun, "Efficient bug detection by inferring implicit API contract of pointer state transition," in *Proceedings of the IEEE International Conference on Dependable Systems and Networks (DSN)*, 2026.
- [31] Ribeiro, Mamede, and Ferreira, "Systematic API testing through model checking and executable contracts," in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2026.
- [32] Alonso, Ernst, Segura, and Ruiz-Cortés, "Test oracle generation for REST APIs," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 35, no. 1, p. art. 19, 2025.
- [33] Cheon *et al.*, "Enhancing design-by-contract with frame specifications," in *Proceedings of the 20th International Conference on Software Technologies (ICSOT)*. SCITEPRESS, 2025.
- [34] Ghosal, Jonsson, and Rümmer, "An active learning approach to synthesizing program contracts," in *Lecture Notes in Computer Science*, vol. 14323. Springer, 2023.
- [35] Ahmed, Cruz *et al.*, "Design by contract for deep learning APIs," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023.
- [36] Segura, Parejo, Troya, and Ruiz-Cortés, "Metamorphic testing of RESTful web APIs," *IEEE Transactions on Software Engineering*, vol. 44, no. 11, 2018.
- [37] Segura, Fraser, Sanchez, and Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Transactions on Software Engineering*, vol. 42, no. 9, 2016.
- [38] Chen, Kuo *et al.*, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys*, vol. 51, no. 1, p. art. 4, 2019.
- [39] Atlidakis, Godefroid, and Polishchuk, "RESTler: Stateful REST API fuzzing," in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, 2019.
- [40] Godefroid, Lehmann, and Polishchuk, "Differential regression testing for REST APIs," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2020.
- [41] Gyori, Shi, Hariri, and Marinov, "Reliable testing: Detecting state-polluting tests to prevent test dependency," in *Proceedings of ISSTA*, 2015.
- [42] Lam, Oei *et al.*, "iDFlakies: A framework for detecting and partially classifying flaky tests," in *Proceedings of ICST*, 2019.
- [43] Liu, Peng *et al.*, "ToolGate: Contract-grounded and verified tool execution for LLMs," 2026.
- [44] Bhardwaj, "Agent behavioral contracts: Formal specification and runtime enforcement for reliable autonomous AI agents," Zenodo, 2026, preprint, not peer-reviewed.
- [45] Babu and Iyer, "Contract2Tool: Learning preconditions and effects for reliable tool-augmented LLM agents," 2026.
- [46] Wang, He *et al.*, "ContractBench: Can LLM agents preserve observation contracts?" 2026.
- [47] Shah, Morovati, Rahman, and Khomh, "Characterizing faults in agentic AI: A taxonomy of types, symptoms, and root causes," 2026.
- [48] Ning, Zhang *et al.*, "Defining and detecting the defects of large language model-based autonomous agents," *IEEE Transactions on Software Engineering*, 2026.
- [49] Ma, Feng *et al.*, "MM-ToolSandBox: A unified framework for evaluating visual tool-calling agents," 2026.
- [50] Jia and Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.